

TransRetarget : A Human-Robot Hand Motion Retargeting Framework Based on Temporal-Spatial Transformer

Ziyan Liang, Qing Gao*, Yuanchuan Lai, Zhaojie Ju

Abstract—Human-robot motion retargeting is a technology that maps human movements to robot movements. It has emerged as a pivotal function in embodied intelligence, particularly in domains involving interaction and grasping, such as the teleoperation of dexterous robotic hands. Most of the existing models mainly focus on single-frame retargeting, often ignoring the inter-frame dependencies, which results in the generated robot retargeting outcomes lacking smoothness. In this paper, we propose TransRetarget, a novel multi-frame human-robot hand motion retargeting method based on a temporal-spatial Transformer architecture. By leveraging the powerful modeling capabilities of Transformers, our method can effectively capture the complex relationships between human hand poses and robotic hand actions. It can not only comprehensively model the intra-frame joint correlations, but also capture inter-frame temporal dependencies, thereby generating high-quality robotic hand motion data. Furthermore, to ensure action consistency between human hand poses and robotic hand actions, we design a suite of loss functions, including finger side movement, finger flexion, and fingertip distance, which are integrated into a composite loss function for training. Experimental results demonstrate that TransRetarget achieves superior performance across multiple datasets and various dexterous robotic hand platforms. The source code was released on <https://github.com/ZaynLeung/TransRetargetv1>.

I. INTRODUCTION

Human-robot motion retargeting refers to the process of mapping human motion sequences in the source domain to robots in the target domain with different physical structures, kinematic constraints, and degrees of freedom configurations, either in online or offline. Online human-robot motion retargeting is the core of robot teleoperation technology, primarily used in scenarios requiring precise robotic manipulation, such as remote surgery. Offline human-robot motion retargeting, on the other hand, is mainly used in robot data acquisition scenarios, a field that has become increasingly popular in recent years with the development of embodied artificial intelligence [1] [2]. For a robot, the part that is mainly responsible for interaction is the robotic hand [3]. Different models of robotic hands have structural differences, which will affect the outcome of retargeting. An

This work was supported in part by the Shenzhen Science and Technology Program under Grant ZDCY20250901100201002, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2025A1515011954, 2023A1515110074.

Ziyan Liang, Qing Gao, Yuanchuan Lai, are with the School of Electronics and Communication Engineering, Sun Yat-sen University, Shenzhen, China (e-mail: liangzysysu@163.com; gaoqing2@mail.sysu.edu.cn; 2865326869lai@gmail.com).

Zhaojie Ju is with the School of Computing, University of Portsmouth, Portsmouth PO1 3HE, UK (email: Zhaojie.Ju@port.ac.uk).

*Corresponding author: Qing Gao, gaoqing2@mailsysu.edu.cn.

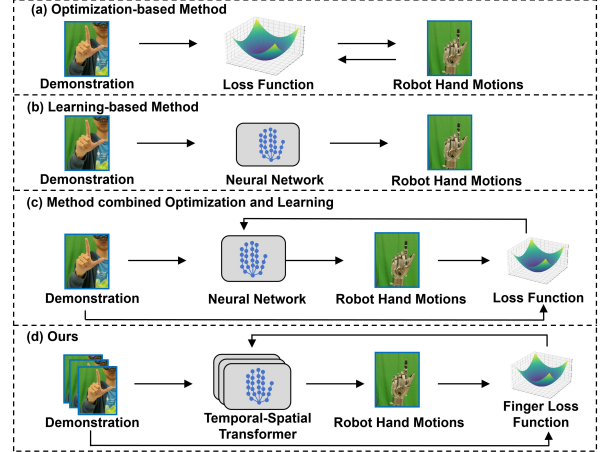


Fig. 1: Illustration of different motion retargeting methods. (a) Optimization-based method; (b) Learning-based method; (c) Method combining optimization and learning; (d) Ours.

ideal method should be platform-compatible, adaptable to various models of dexterous hands, so as to facilitate use by users from different manufacturers. Also, because the robotic hand has more degrees of freedom, the hand motion retargeting method tends to be more complex than the motion retargeting method used for arms. Therefore, developing an accurate, efficient, and generalized human-robot hand motion retargeting framework has become a crucial research challenge.

In recent years, numerous studies have been dedicated to addressing the aforementioned challenges. Some studies use optimization-based method, as shown in Fig. 1(a), such as a research that uses unified kinematic mapping for real-time optimization, which has a fast speed and good platform generalization ability, but lacks in accuracy due to simplified mathematical model and local solution [4]. Some researchers have turned to learning-based methods, as shown in Fig. 1(b), which improve retargeting accuracy but lack speed and platform generalization capabilities by increasing model parameter capacities [5]. In order to improve both retargeting speed, accuracy, and platform generalization, studies have proposed methods combining optimization and learning [6], as shown in Fig. 1(c). However, in the above methods, single-frame data is solely used for retargeting, without extracting the inter-frame correlation information of the continuously input frames during its operation. As a result, the generated robot action sequence lacks smoothness.

To address the aforementioned issues, we need to develop a method to extract the temporal-spatial features within the

input action sequence. As shown in Fig. 1(d), the main difference between the aforementioned methods and ours lies in the introduction of temporal-spatial features. Facing the challenge of temporal-spatial feature extraction in hand motion retargeting, we draw inspiration from the Transformer architecture, as it has been studied in the context of human bone sequences because of its efficiency, scalability, and formidable modeling capacity [7]. In order to enhance the naturalness of the generated robotic hand motions, we designed a loss functions that consists of multiple items to be used during the training process. The loss function is designed based on the aim of making the vector representations of the finger parts of the human hand and the robotic hand similar. Therefore, we made the choice of introducing the Transformer architecture to model the temporal-spatial correlations of human hand motion data, providing a novel method for improving the smoothness and accuracy of motions.

Our contributions are primarily summarized in the following aspects:

- **A Human-Robot Hand Motion Retargeting Framework based on Temporal-Spatial Transformer:** We proposed a retargeting method using the Transformer architecture, which extracts the temporal-spatial features of the hand motions through the temporal-spatial encoder. This method can effectively capture the inter-frame relationship between human hand postures and robotic hand motions.
- **A Multi-Loss Function System for Self-Supervised Learning:** We have developed a comprehensive multi-loss function system. By encouraging the similarity of the finger vectors during training, we enhanced the plausibility of the generated robotic hand motions.

II. RELATED WORK

A. Human-Robot Motion Retargeting

1) *Optimization-Based Retargeting Methods:* These methods treat motion retargeting as a constrained optimization problem [8] [9]. The core logic involves using a mathematical solver to find a set of robot joint angles in each frame that minimizes the distance between the robot's end effector and the human keypoint. The advantage of this method lies in the fact that it does not require training data and is platform-agnostic. However, its disadvantages include being prone to getting stuck in local optimal solutions, resulting in unstable output, and the accuracy depends on the number of iterations and is often relatively low.

2) *Learning-Based Retargeting Methods:* These methods adopt a data-driven approach, using deep neural networks to directly establish an end-to-end mapping from the human motion space to the robot joint space [10] [11]. There are also studies that use imitation learning to map human motions onto the robot work space [12]. Their biggest advantage is extremely fast inference speed, enabling very high-frequency real-time responses. Furthermore, because the network learns

prior knowledge of human motions, it exhibits stronger stability when handling noisy data and severe occlusion. The learning method improves the accuracy of the retargeting results, but reduces the platform's generalization and needs more inference time.

3) *Methods Combined with Optimization and Learning:*

These methods aim to combine the advantages of the previous two approaches, such as reaserch employing a latent space optimization assisted by neural network initialization architecture [13] [14]. The method maps human motions to the latent space and performs iterative optimization within the space [15] [6]. There have also been studies that, after improving the graph neural network, used more flexible graph structures to complete the human-robot motion retargeting task [16] [17]. This logic solves the problem of pure optimization methods relying on good initial values and also compensates for the shortcomings of pure learning methods in terms of inference time and platform generalization, making it the mainstream research direction in the field of robotic hand teleoperation. However, this method still has issues in terms of motion continuity and smoothness, because it only uses data frame-by-frame to achieve motion retargeting. Therefore, we need to propose a method that can extract temporal-spatial features from multiple frames of data.

B. From Transformer to TransRetarget

Transformer has its core components, including positional encoding, scaled dot-product attention, multi-head self-attention (MHSA), feedforward neural networks (FFN), and residual connection with Layer Normalization [18] [19]. Positional encoding injects temporal or spatial position information into the input vectors, compensating for the loss of sequential sense when the model processes in parallel. Dot-Product Attention measures the correlation between elements by calculating the inner product of the query vector and the key vector, and introduces a scaling factor to stabilize the gradient, ultimately achieving adaptive weighted aggregation of the value vector. MHSA enables the model to simultaneously capture multi-dimensional feature relationships ranging from local details to global semantics. And FFN ensures the stability of the numerical distribution while effectively alleviating the problem of gradient disappearance in the deep architecture.

Due to its powerful long-distance dependency modeling and parallel sequence processing capabilities, Transformer is highly suitable for processing sequences of human skeletal point positions. In the field of human pose estimation, Transformer has been widely used for tasks such as pose estimation, action recognition, and motion prediction. Some researches use a Transformer-based architecture to estimate 3D human pose from monocular images [20] [21]. A research uses Transformer for motion recognition [22]. And some research use Transformer for prediction of motion sequences [23]. Since the human skeleton can be viewed as a spatial topological structure or a temporal sequence of motion, researchers use Transformer to model the spatial correlations

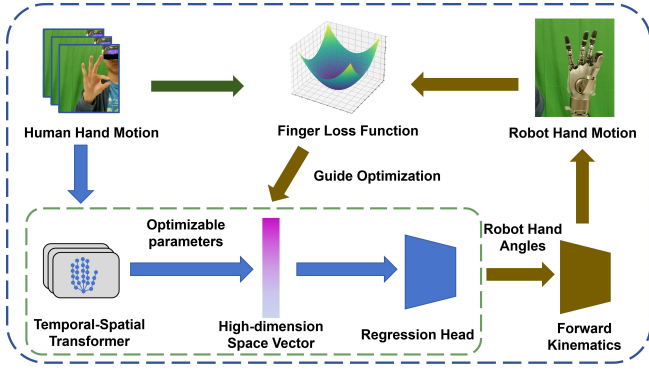


Fig. 2: Overview of TransRetarget framework. The model input is the position of human joints. During the training stage, after the angles are output, they will be converted into the position of the robotic hand through the forward kinematics module, and then enter the loss function to guide the optimization of network parameters, while the data flows along the blue arrows and the brown arrows. In the inference process, the angles are directly used to drive the robotic hand, while the data flows along the blue arrows.

between joints and temporal coherence. These studies also demonstrate the significant advantages of Transformer in processing human skeleton data with complex topological and temporal features.

While Transformer has achieved great success in human pose extraction and temporal prediction, it has not yet been applied to the specific task of human-robot motion retargeting. Most existing retargeting works still rely on frame-by-frame data, often ignoring temporal context information in motion sequences [24] [25]. This leads to disjointed motion when processing complex, nonlinear, heterogeneous mappings [2]. Based on this, we propose using TransRetarget for motion retargeting. By leveraging its powerful sequence modeling capabilities, it learns the continuous mapping from human pose flow to robot joint space, thereby achieving higher performance retargeting results with more temporal smoothness.

III. METHOD

A. Framework

Our model proposes a human-robot hand motion retargeting method based on the Transformer architecture. As shown in Fig. 2, the system first captures 3D human hand pose data using a VR headset or a camera, from which 3D hand keypoint positions are extracted via a pose estimation algorithm. In our experiment, we used VisionPro as the VR headset, Realsense as the camera, and the pose estimation algorithm was SLAMHR [26]. Following a data preprocessing stage, the input data of the current frame is concatenated with preceding frames and fed into the Transformer network. The data passes sequentially through a spatial Transformer module and a temporal Transformer module for encoding. During the training phase, the encoded data is processed through a forward kinematics (FK) module to derive robotic joint positions, after which the loss functions are calculated for gradient descent. In the inference stage, the robotic joint angles are directly generated via a regression head to achieve real-time retargeting.

1) *Core Computational Unit:* We design a Transformer network as the main module, as shown in Fig. 3. The Transformer encoder is composed of multiple layers of basic blocks stacked together. The core computational unit of this method utilizes a standard Transformer block, comprising multi-head self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \quad (1)$$

$$\text{MHSA}(Q, K, V) = \text{Concat}(H_1, H_2, \dots, H_h)W \quad (2)$$

$$\text{where } H_i = \text{Attention}(Q_i, K_i, V_i), i \in [1, h],$$

with scaled dot-product attention and a feed-forward network. Training stability is ensured through residual connections and layer normalization. Here, $Q, K, V \in \mathbb{R}^{N \times d}$, where N represents the number of vectors in the input sequence and d denotes the dimension of the input vectors. To prevent the dot-product results from growing excessively large, in our method we apply a scaling factor of $\sqrt{d}$. Self-attention is computed independently within each subspace, and the results are concatenated to produce the final output. The dimension of each head is defined as $d_h = d/h$, where h is the number of heads.

2) *Input Preprocessing and Normalization:* We take the 3D human hand pose as input. Each single-frame data consists of the output from VisionPro or a pose estimation algorithm [26], specifically the 3D positions of hand keypoints, which are normalized to the wrist coordinate system prior to input. To align the orientation of the hand data uniformly, we use three points on the palm surface to construct a plane, and align the normal vector of this plane with the x-axis. This ensures the feasibility of subsequent calculations of the loss function.

3) *Patch Embedding and Positional Encoding:* Each input is treated as a patch, following the terminology used in vision Transformer [27] [28]. For an input sequence $X \in \mathbb{R}^{f \times J \times 3}$, f denotes the number of frames in the sequence, J represents the number of joints, and 3 corresponds to the coordinate dimensions of each joint. The sequence is defined as $X = \{x_1, x_2, \dots, x_f\}$, where $x_i \in \mathbb{R}^{J \times 3}$ represents the positions of all joints in the i -th frame and can be regarded as the input vector for that frame. Subsequently, each patch of the input sequence is embedded into a high-dimensional feature space. During this process, the network utilizes positional embeddings to preserve the positional information of the sequence. This stage employs a linear projection matrix $E \in \mathbb{R}^{J \times 3 \times C}$ combined with positional embeddings $E_{pos} \in \mathbb{R}^{f \times C}$ to derive the initial feature representation Y_0 . A learnable parameter matrix is used as the position embedding here. After the input sequence $X \in \mathbb{R}^{f \times J \times 3}$ undergoes patch embedding and encoded with positional embeddings, we obtain the feature representation $Y_0 \in \mathbb{R}^{f \times J \times C}$, where C is the embedding dimension. Y_0 is then encoded through the temporal Transformer. Similarly, following the application of temporal positional embeddings and processing through multiple Transformer blocks, the encoded feature representation $Z_N \in \mathbb{R}^{f \times J \times C}$ is obtained.

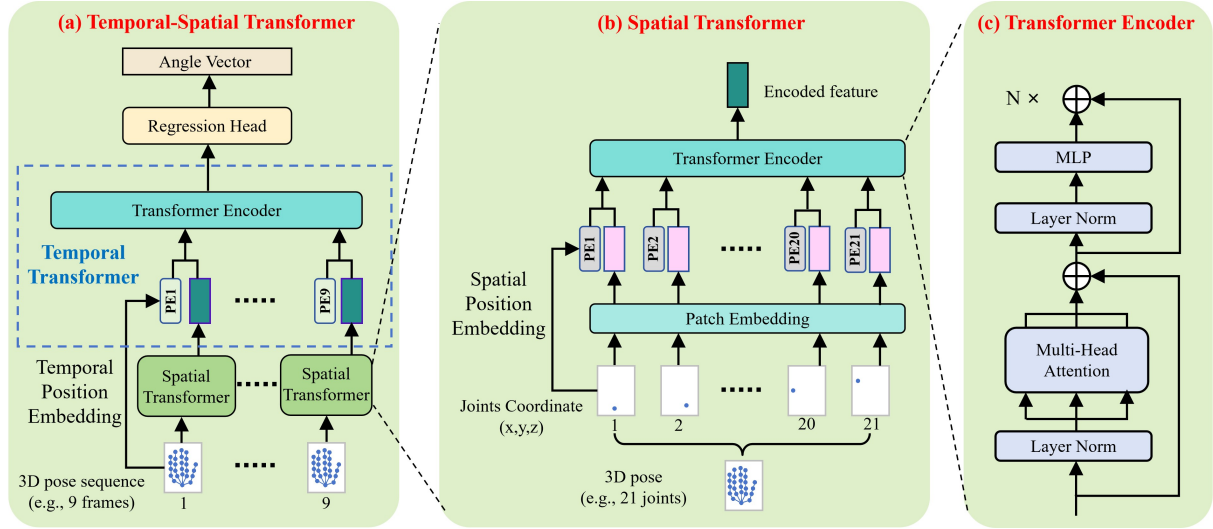


Fig. 3: Temporal-Spatial Transformer structure. The input is a sequence of 3D human hand poses, which are first processed through a spatial transformer to extract inter-joint features, and then fed into a temporal transformer to capture the temporal dynamics across frames. Finally, the output is regressed into robotic joint angles. (a) Temporal-Spatial Transformer module; (b) Spatial Transformer module; (c) Transformer encoder.

Z_N is the vector obtained by Z_0 after passing through N layers of spatial Transformer blocks.

4) *Regression Head and FK Module*: After obtaining the time features of multiple frames, we integrate them and then map them to the joint space that matches the robotic hand through a multi-layer perceptron block as the regression head to calculate hand joint angles $\vec{\theta} \in \mathbb{R}^{1 \times (J' \cdot 1)}$. During the evaluation stage, we use a fixed-parameter FK module to derive the 3D coordinates of the robotic hand from the joint angles. The process is as follows:

$$P = P_{parent} \cdot T_{offset} \cdot R(\theta, \vec{n}). \quad (3)$$

Here, P_{parent} is the parent joint position, T_{offset} is the offset matrix, $R(\theta, \vec{n})$ is the rotation matrix, θ is the rotation angle, and $\vec{n}$ is the rotation axis of the joint. Then we calculated the losses between the human hand 3D positions and the robotic hand 3D positions to optimize the model parameters.

B. Temporal-Spatial Transformer Architecture

The **spatial Transformer module** is designed to extract spatial features from the input sequence with a trainable linear projection to map the coordinate of each joint to a high dimension from individual frames. For an input sequence $X \in \mathbb{R}^{f \times J \times 3}$, f denotes the number of frames in the sequence, J represents the number of joints, and 3 corresponds to the joint coordinate dimensions. The sequence is defined as $X = \{x_1, x_2, \dots, x_f\}$, where $x_i \in \mathbb{R}^{J \times 3}$ denotes the joint positions in the i -th frame, serving as the input vector for that frame. The patch embedding E is the trainable linear projection layer that embeds each vector of the input sequence into a high-dimensional feature space $\mathbb{R}^{J \times C}$. We refer to this process as spatial patch embedding. The resulting embeddings are combined with learnable spatial positional embeddings $E_{pos} \in \mathbb{R}^{J \times C}$. Consequently, the input vector $x_i \in \mathbb{R}^{J \times 3}$ is embedded as $y_i \in \mathbb{R}^{J \times C}$. During this procedure, the Transformer network utilizes positional

embeddings to retain the spatial structural information of the sequence. This process can be expressed as:

$$Y = [x_1 E; x_2 E; \dots; x_f E] + E_{pos}. \quad (4)$$

The final joint features $y_i \in \mathbb{R}^{J \times C}$ are then fed into the spatial Transformer encoder for encoding. This encoder utilizes the self-attention mechanism to allow each joint to interact globally with all other nodes, thereby integrating information from the entire skeletal structure. For the input vector y_i of the i -th frame, the spatial encoding y_{iL} is obtained after passing through a spatial Transformer encoder with L layers. Subsequently, y_{iL} is passed to the temporal Transformer for further encoding.

The **temporal Transformer module** is designed to model temporal features across frames, while the spatial Transformer module encodes high-dimensional inter-joint features within a single frame. For the i -th frame, each spatial representation y_{iL} is flattened into a vector $y^i \in \mathbb{R}^{1 \times (J \cdot C)}$. Subsequently, the set of vectors y^i from f adjacent frames is concatenated to form the temporal sequence $y^f \in \mathbb{R}^{f \times (J \cdot C)}$. Prior to entering the temporal encoder, we incorporate a learnable temporal positional embedding matrix E_{tpos} to preserve the chronological information of the sequence. The temporal Transformer encoder adopts an architecture identical to that of the spatial Transformer encoder, consisting of MHSA mechanisms and MLP networks. After processing through encoder layers, the temporal Transformer encoder outputs the final temporal features $Z_N \in \mathbb{R}^{f \times (J \cdot C)}$. Z_N is the vector obtained by Z_0 after passing through N layers of spatial Transformer blocks.

C. Finger Loss Function

Prior to calculating the losses, we normalize both the human hand data and the robotic hand data to ensure their scales are comparable. Specifically, we project the keypoint positions of both the human and robotic hands into a unified

coordinate system before performing further computations. To encourage the robotic hand motions to closely resemble human demonstrations while remaining kinematically feasible, we define four loss functions: the finger vector loss L_{fv} , the finger tip vector loss L_{tv} , the fingertip distance loss L_{td} , and the collision loss L_{col} . The weights for these loss components are pre-configured, with λ_{fv} , λ_{tv} , λ_{td} , and λ_{col} set to 500, 500, 50, and 10, respectively. These weights are empirical values tuned to balance the magnitude of different loss components and ensure numerical stability during the joint optimization process. The overall loss function is defined as follows:

$$L_{total} = \lambda_{fv} \cdot L_{fv} + \lambda_{tv} \cdot L_{tv} + \lambda_{td} \cdot L_{td} + \lambda_{col} \cdot L_{col}. \quad (5)$$

1) *Finger Vector Loss: (L_{fv})* The finger vector loss L_{fv} encourages the lateral movement of the robotic finger roots to align with those of the human hand. It utilizes the mean squared error (MSE) to measure the discrepancy between normalized vectors. During computation, human finger root vectors $\mathbf{v}_i$ and robot finger root vectors $\hat{\mathbf{v}}_i$ are derived from the proximal interphalangeal (PIP) joints and metacarpophalangeal (MCP) joints of both the human and robotic hands; these vectors are subsequently normalized before calculating the MSE:

$$L_{fv} = \frac{1}{N} \sum_{i=1}^N \left\| \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|} - \frac{\hat{\mathbf{v}}_i}{\|\hat{\mathbf{v}}_i\|} \right\|^2. \quad (6)$$

2) *Finger Tip Vector Loss: (L_{tv})* The finger tip vector loss L_{tv} encourages the robotic hand to be similar to the human hand in terms of the vectors formed by the fingertips and the MCP joints. This part of the vectors is closely related to the finger grasping function. During the calculation process, in order to keep the positional relationship between the fingertips and the palm consistent, we subtract the position of the wrist from the position of each fingertip to calculate the human fingertip vector $\mathbf{t}_i$ and the robotic fingertip vector $\hat{\mathbf{t}}_i$ of each finger, and then normalize them individually to eliminate the scale differences before calculating the MSE:

$$L_{tv} = \frac{1}{N} \sum_{i=1}^N \left\| \frac{\mathbf{t}_i}{\|\mathbf{t}_i\|} - \frac{\hat{\mathbf{t}}_i}{\|\hat{\mathbf{t}}_i\|} \right\|^2. \quad (7)$$

3) *Finger Tip Distance Loss: (L_{td})* The finger tip distance loss L_{td} ensures that the relative distances between robotic fingertips match those of the human hand. It uses MSE to evaluate the similarity of distances between specific pairs of fingertips:

$$L_{td} = \frac{1}{N} \sum_{i=1}^N \left\| \mathbf{d}_i - \hat{\mathbf{d}}_i \right\|^2. \quad (8)$$

When conducting the actual calculation, we only measured human hand tips distances d_i and robotic fingertip distances $\hat{d}_i$ between the thumb and the other four fingers, as well as the distance between adjacent fingertips, because introducing too many distance calculations may make parameter optimization difficult. And the above-mentioned relationships between the fingers have a significant impact on the interaction effect of the robotic hand.

4) *Collision Loss: (L_{col})* The collision loss L_{col} aims to penalize motion angles that result in self-collision of the robotic hand. We model each joint as a sphere and calculate the inter-joint distances $c_{i,j}$ to prevent collisions between robotic finger segments. A penalty term is generated based on a minimum distance threshold between these modeled spheres. Considering that the collision distance is usually a very small value, we consider using the negative logarithm function for calculation. In this way, as the distance decreases, the loss value will increase sharply [29]:

$$L_{col} = \sum_{c_{i,j} < c_{min}} -\log(c_{i,j}). \quad (9)$$

IV. EXPERIMENTS

A. Experimental Setup

1) *Robotic Hand Experimental Platforms:* To evaluate the deployability of our TransRetarget on different robotic hands platforms, we conducted human hand motion retargeting experiments on three robotic hands: Linker, Shadow, and Inspire. The Linker hand was tested in both physical and virtual environments, while Shadow and Inspire were tested in the virtual environment. The Linker robotic hand has 17 degrees of freedom. The Shadow robotic hand has 23 degrees of freedom. The Inspire robotic hand has 13 degrees of freedom.

2) *Dataset Composition and Preprocessing:* Training and evaluation used two datasets. The first is the human hand motion dataset (VP dataset) collected by us using Vision-Pro, which mainly consists of random hand movements performed by one volunteer wearing a head-mounted device, and 10,000 frames were collected within 10 minutes, providing position data for 25 key points of each hand. The second is the CSL-Daily sign language image dataset [30], where 6,000 frames of data were extracted by Slahmr from single-frame images to construct the three-dimensional hand posture [26]. Actions included 299 randomly selected sign language actions demonstrated by three volunteers, each action lasting five to forty frames, and each frame provided position data for 16 key points of each hand. Both of these datasets record complex human hand movements and cover a wide range of daily hand actions.

3) *Hardware Environment and Training Specifications:* Our work is implemented using PyTorch, and is trained using the Adam optimizer on an NVIDIA RTX 3050 Laptop GPU and an Intel Core i7-12700H CPU, with learning rate of 1e-4. Batch size for all training sessions is 1024. At the end of each epoch, new indices will be selected for training.

B. Comparison Experiments

We conducted comparative experiments to evaluate the performance of our method against two baseline approaches. We also present a comparison of the retargeting errors across different models in the Table I.

In our study, experiments were conducted on our self-built dataset and the CSL-Daily dataset. We selected NLO [13] and VMR [31] as baselines. In order to evaluate the

interaction performance of the robotic hand, we directly used the difference in the end points of the human hand and the robotic hand as the evaluation criterion [32], using the FK module to calculate the fingertip positions, and finally used MSE to calculate the positional difference of the fingertips when both were scaled to the same scale. As shown in Table I, the error of the fingertips positions is lower than that of the comparison methods. Our proposed method outperforms the other two reference methods across all datasets and fingers, demonstrating the effectiveness of our approach.

TABLE I: Comparison of Hand Retargeting Performance.

Tip Error(m)	VP dataset			CSL-daily		
	NLO	VMR	Ours	NLO	VMR	Ours
Thumb	0.0931	0.0932	0.0439	0.0524	0.0518	0.0255
Index	0.0689	0.0686	0.0326	0.0465	0.0459	0.0344
Middle	0.0516	0.0509	0.0300	0.0464	0.0459	0.0319
Ring	0.0494	0.0496	0.0316	0.0459	0.0450	0.0324
Pink	0.0569	0.0573	0.0423	0.0508	0.0503	0.0404

C. Ablation Experiments

To verify the contribution of different components in the finger loss of our model, we visualized MHSA and conducted finger loss function ablation experiments.

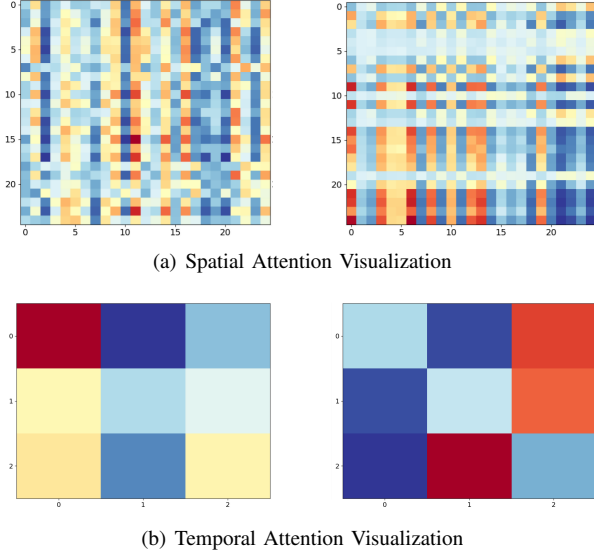


Fig. 4: Attention visualization. Part (a) is spatial attention visualization. Part (b) temporal attention visualization. The attention weights of the spatial and temporal self-attention mechanism are visualized, where the horizontal represent the input labeled as keys, vertical axes represent the output labeled as queries. From left to right, they are head 0 and 1. The color intensity indicates the attention weight. More red color indicates higher attention weight, while more blue color indicates lower attention weight and the attention values have been normalized to the range of 0 to 1, respectively.

1) *Attention Visualization*: To illustrate the role of the attention mechanism in MHSA module, we evaluated specific actions in the dataset and visualized the attention. As shown in Fig. 4, we visually demonstrated the attention of the MHSA mechanism for both space and time [20].

Regarding the spatial attention map, we observed that the attention heads output different attention weights when processing different action sequences, and the weights are related to the joint numbers. These weights reflect various local relationships between the input joints, indicating that the model can capture the coordinated movement patterns between fingers. For example, in Fig. 4 (a), we selected the first two attention heads of the spatial encoder. We can see that in the spatial attention head two on the right side, it mainly takes responsibility for connecting other joints with joint 0, 3, 6, 8, 12, 13, and 19.

As for the temporal self-attention map, our observation indicates that when processing consecutive frame data, the attention heads mainly focus on the input of the latest frame. As shown in Fig. 4 (b), we selected the first two attention heads of the first-level temporal encoder, where the zeroth frame represents the current frame and the first frame represents the previous frame. We can observe that the head 0 of the left side of Fig 4 (b), outputs are mainly related to the current frame. However, head 1 on the right side also shows a connection with the previous frames. This indicates that the model is capable of handling consecutive sequences of action frames and can effectively consider the relationships between joint nodes as well as the relationships between input frames.

2) *Loss Function Ablation Experiments on VP Dataset*: We conducted an ablation experiment based on the VP dataset to evaluate the impact of each component of the loss function on the model performance and its overall effectiveness. Our loss function consists of four parts: finger swing, fingertip position, fingertip vector, and collision loss. After removing the corresponding loss function part, we used the same fingertip normalization position comparison method as in the previous experiments for evaluation, and calculated the speed and acceleration of the fingertip to make a comparison. Finally we calculated the average error of the results. Additionally, we used Gym to evaluate whether the generated motions caused any collisions [33]. In the last column of Tab II, we list the number of frames that experienced collisions. The total number of frames we tested was 2400. If the proportion of collision frames was less than 99%, we concluded that there would be no self-collision. In the ablation experiments, we successively set the weights of each part of the loss function to zero in turn, and then carried out training and testing. After removing the collision loss, the retargeting results showed frequent collisions in our tests. When the other three parts of the loss were not used, the angle quality of the retargeting results was inferior to the original results.

D. Generalization to Different Robotic Hands

The cross-platform generalization capability of our TransRetarget hand motion retargeting method was visualized on three robotic hand platforms: Linker, Shadow, and Inspire. As shown in Fig. 5, in both simulation and real-world environments, a consistent set of human hand motion samples was used, demonstrating stable reproduction quality

TABLE II: Ablation Experiment Results.

Methods	Tip Position Error (m)	Velocity Error (m/s)	Acceleration Error (m/s ²)	Collision Occurred (frames)
Ours (Full Losses)	0.0361	0.0011	0.0018	18
w/o Finger Root Vector Loss	0.0369	0.0012	0.0021	20
w/o Finger Tip Vector Loss	0.0450	0.0016	0.0029	14
w/o Tip Distance Loss	0.0371	0.0012	0.0022	26
w/o Collision Loss	0.0374	0.0011	0.0019	143



Fig. 5: Performance comparison of different robotic hands. From left to right, the five lines represent: (a) Human hand; (b) the Linker robotic hand in the simulated environment; (c) the Shadow robotic hand in the simulated environment; (d) the Inspire robotic hand in the simulated environment; (e) the Linker robotic hand in the physical environment.

across platforms. In addition to visualization, we conducted quantitative evaluations on the VP-dataset using fin angle, as summarized in Table III. The evaluation method is the same as that in the previous subsection. The results show that the framework maintains stable hand retargeting quality across different platforms.

TABLE III: Performance Comparison of Different Hands.

Metrics	Linker Hand	Shadow Hand	Inspire Hand
Tip Position Error (m)	0.0361	0.0266	0.0358
Velocity Error (m/s)	0.0011	0.0013	0.0011
Acceleration Error (m/s ²)	0.0018	0.0023	0.0019
Collision Occurred	×	×	×

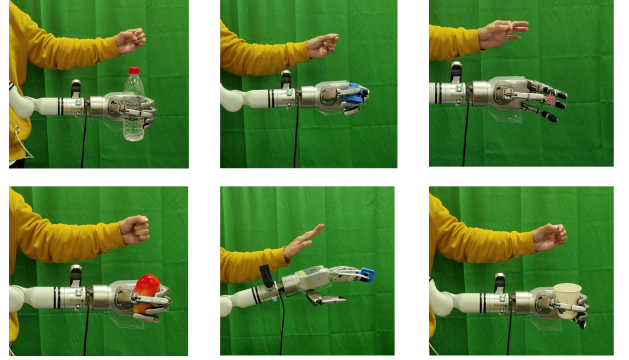


Fig. 6: Grasping performance evaluation. In our evaluation, we selected six objects that could be grasped by human hands for the experiment. In this figure, from left to right, then from the top to the bottom, the objects are a water bottle, a plastic bolt, a binder clip, a plastic mango, a plastic nut, and a paper cup.

E. Grasping Performance Evaluation

To demonstrate the practicality of our method in object interaction, we conducted some experiments on object grasping. As shown in Fig. 6, we selected six objects that could be grasped by human hands for the experiment. We counted the number of successful grasps and the time consumed for the grasping actions. The results are presented in Table IV. We conducted 20 grasping tests on the same object and counted the number of successful attempts to calculate the success rate. At the same time, based on the time elapsed from the start of the operation to when the object was stabilized for grasping, we measured the average time consumed for grasping different objects. We achieved a high success rate on objects like water bottles that are easy to grasp, but the success rate for objects with small contact points like binder clips needs to be improved. In addition, the performance of real-time retargeting was evaluated. In our experiment, the system achieved a frame rate of 60 frames per second. Among them, VisionPro took an average of 0.003 seconds to process each frame of human hand pose, while our retargeting module completed the processing of each frame on average in 0.007 seconds. Due to the fact that VisionPro uses a wireless network connection and the robotic hand needs time to process the instructions sent by the host, there are hardware-related delay of averagely 0.05 seconds.

V. CONCLUSION

This study proposes a cross-platform real-time motion retargeting framework that integrates temporal-spatial information. Our work is the first to apply the Transformer

TABLE IV: Grasping Performance Evaluation

Grasping Objects	Success Rate	Execution Time(s)
Water Bottle	18/20	3.17
Plastic Bolt	17/20	1.53
Binder Clip	15/20	4.50
Plastic Mango	19/20	1.60
Plastic Nut	16/20	1.43
Paper Cup	16/20	2.80

architecture to the task of hand motion retargeting between humans and robots, and the model training is carried out through the loss function designed by us. The experimental results on different datasets show that this framework can effectively capture the topological features of human hand motions and robot motions. By inputting the 3D position of human hand motions, it can generate the output of high-quality robotic hand motion. Through ablation experiments, it was proved that each part of the loss in our model has a positive effect on the output result. Additionally, this framework has been verified on multiple dexterous robotic hands.

When introducing multi-frame sequences with longer contexts leads to excessive consumption of computing resources, we believe that this can be addressed by optimizing the encoder to improve efficiency. In addition, tactile information can also be incorporated into our retargeting model.

REFERENCES

- [1] A. O'Neill, A. Rehman, A. Maddukuri *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.
- [2] A. Brohan, N. Brown, J. Carbajal *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.
- [3] S. Li, N. Hendrich, H. Liang *et al.*, "A dexterous hand-arm teleoperation system based on hand pose estimation and active vision," *IEEE Transactions on Cybernetics*, vol. 54, no. 3, pp. 1417–1428, 2022.
- [4] Y. Qin, W. Yang, B. Huang *et al.*, "Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system," *arXiv preprint arXiv:2307.04577*, 2023.
- [5] X. Figuera, S. Park, and H. Ahn, "Redefining data pairing for motion retargeting leveraging a human body prior," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 4800–4807.
- [6] Y. Lai, Q. Gao, Z. Liang *et al.*, "Dextele: A dual-arm dexterous teleoperation system based on motion retargeting and adaptive force control," in *2026 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2026.
- [7] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [8] J. Orbik, S. Li, and D. Lee, "Human hand motion retargeting for dexterous robotic hand," in *2021 18th International Conference on Ubiquitous Robots (UR)*. IEEE, 2021, pp. 264–270.
- [9] K. Ayusawa and E. Yoshida, "Motion retargeting for humanoid robots based on simultaneous morphing parameter identification and motion optimization," *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1343–1357, 2017.
- [10] Z.-H. Yin, C. Wang, L. Pineda *et al.*, "Geometric retargeting: A principled, ultrafast neural hand retargeting algorithm," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 17376–17382.
- [11] Y. Yan and D. Lee, "Learning dexterous robot hand control by imitating human hands," in *2025 22nd International Conference on Ubiquitous Robots (UR)*. IEEE, 2025, pp. 307–312.
- [12] X. Cheng, Q. Gao, G. Chen *et al.*, "Viper: Verifiable imitation learning policy for efficient robotic manipulation," in *2026 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2026.
- [13] H. Zhang, W. Li, J. Liu *et al.*, "Kinematic motion retargeting via neural latent optimization for learning sign language," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4582–4589, 2022.
- [14] T. Kim and J.-H. Lee, "C-3po: Cyclic-three-phase optimization for human-robot motion retargeting based on reinforcement learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 8425–8432.
- [15] T. N. Kipf, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [16] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.
- [17] C. K. Joshi, "Transformers are graph neural networks," *arXiv preprint arXiv:2506.22084*, 2025.
- [18] A. Vaswani, N. Shazeer, N. Parmar *et al.*, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [19] M. Geva, R. Schuster, J. Berant *et al.*, "Transformer feed-forward layers are key-value memories," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 5484–5495.
- [20] C. Zheng, S. Zhu, M. Mendieta *et al.*, "3d human pose estimation with spatial and temporal transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 11656–11665.
- [21] Q. Zhao, C. Zheng, M. Liu *et al.*, "Poseformerv2: Exploring frequency domain for efficient and robust 3d human pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 8877–8886.
- [22] N. C. Camgoz, O. Koller, S. Hadfield *et al.*, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10023–10033.
- [23] B. Jiang, X. Chen, W. Liu *et al.*, "Motiongpt: Human motion as a foreign language," *Advances in Neural Information Processing Systems*, vol. 36, pp. 20067–20079, 2023.
- [24] M. Zhang, Q. Gao, Y. Lai *et al.*, "Hr-gcn: 2d-3d whole-body pose estimation with high-resolution graph convolutional network from a monocular camera," *IEEE Sensors Journal*, 2025.
- [25] R. Ding, Y. Qin, J. Zhu *et al.*, "Bunny-visionpro: Real-time bimanual dexterous teleoperation for imitation learning," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 12248–12255.
- [26] V. Ye, G. Pavlakos, J. Malik *et al.*, "Decoupling human and camera motion from videos in the wild," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 21222–21232.
- [27] W. Lei, Y. Ge, K. Yi *et al.*, "Vit-lens: Towards omni-modal representations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 26647–26657.
- [28] S. Mehta and M. Rastegari, "Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer," *arXiv preprint arXiv:2110.02178*, 2021.
- [29] B. Zheng, D. Liang, Q. Huang *et al.*, "Frame-by-frame motion retargeting with self-collision avoidance from diverse human demonstrations," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8706–8713, 2024.
- [30] H. Zhou, W. Zhou, W. Qi *et al.*, "Improving sign language translation with monolingual data by sign back-translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 1316–1325.
- [31] Y. Lai, Z. Ju, and Q. Gao, "Motion retargeting using graph neural network for vision-guided dexterous robot teleoperation," in *2024 17th International Convention on Rehabilitation Engineering and Assistive Technology (i-CREATE)*. IEEE, 2024, pp. 1–6.
- [32] C. Xin, M. Yu, Y. Jiang *et al.*, "Analyzing key objectives in human-to-robot retargeting for dexterous manipulation," *IEEE Robotics and Automation Practice*, 2026.
- [33] J. Panerati, H. Zheng, S. Zhou *et al.*, "Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 7512–7519.